

Unless a question says otherwise, you may use `List.map`, `List.fold_left`, `List.fold_right`, `List.rev` and `@` (list append).

- Write the types of the following OCaml expressions / definitions. (For a definition `let f ... = ...`, write the type of `f`.)
 - `[(1, "a"); (2, "b")]`
 - `fun f x -> f (f x)`
 - `let f x y = if x = y then [x] else [y]`
 - `(fun x y -> x :: y) 1`
 - `fun (x, y) -> (y, x)`
 - `List.fold_left (fun acc x -> acc ^ x) ""`
 - `fun x -> match x with Some y -> y +. 1.0 | None -> 0.0`
 - ```
let rec g l =
 match l with
 | [] -> None
 | (a, b) :: t -> if b then Some a else g t
```
  - `let compose f g = fun x -> f (g x)`
- Provide expressions (without any explicit type annotations) that have the following types. That is, OCaml should be able to infer exactly the given type from your expression.
  - `int -> int -> int list`
  - `('a -> 'b) -> 'a list -> 'b option`
  - `'a -> ('a -> 'a) -> 'a`
  - `(int * string) list -> string list`
  - `'a option -> 'a -> 'a`
  - `bool -> (int -> int) -> int -> int`
  - `'a list -> 'a list -> bool`
- Give the static semantics (type-checking rule) and the dynamic semantics (evaluation rule) for each of the following, in the inference-rule notation used in class. Use  $e : t$  for “ $e$  has type  $t$ ” and  $e \rightarrow v$  for “ $e$  evaluates to  $v$ ”.
  - The short-circuiting conjunction `e1 && e2`. (Recall that `e2` is evaluated only when `e1` evaluates to `true`, so you will need more than one dynamic rule.)
  - The list cons expression `e1 :: e2`.
  - Record construction `{f1 = e1; ...; fn = en}`, for the record type
 

```
type r = {f1 : t1; ...; fn : tn}
```
- For each of the following programs, if the program does not type-check then give a short and specific error message explaining why. Otherwise, state the value that `ans` is bound to after the program executes. For example, for the program
 

```
let f1 (x,y) = if x then y + 1 else x
```

a good answer would be: “the then-branch has type `int` while the else-branch has type `bool`”.

- ```
let x = 3
let f y = x + y
let x = 10
let ans = f x
```

(b) **type** `pt` = {`x` : `int`; `y` : `int`}

```
let p = {x = 1; y = 2}
let q = {p with y = 5}
let ans = p.y + q.y
```

(c) **type** `shape` = `Sq of int` | `Tri of int * int`

```
let area s =
  match s with
  | Sq a -> a * a
  | Tri (b, h) -> b *. h /. 2.0

let ans = area (Sq 3)
```

(d) **let rec** `f` `l` =
 match `l` **with**
 | [] -> []
 | [x] -> x
 | x :: t -> x :: f t

```
let ans = f [[1]; [2]]
```

(e) **exception** `Neg of int`

```
let rec sum l =
  match l with
  | [] -> 0
  | h :: t -> if h < 0 then raise (Neg h) else h + sum t

let ans = try sum [1; 2; -3; 4] with Neg n -> n
```

(f) **let** `apply_twice f` = **fun** `x` -> `f (f x)`
 let `ans` = (`apply_twice (fun s -> s ^ " !")`) "hi"

5. This problem uses the following variant type, which models a (simplified) file system. A `File` carries its name and its size in bytes; a `Dir` carries its name and the list of entries it contains.

```
type fs =
  | File of string * int
  | Dir of string * fs list
```

- Give a value of type `fs` representing a directory named "root" which contains a file "a.txt" of size 10 and a sub-directory "sub", which in turn contains a single file "b.txt" of size 20.
- Define `total_size : fs -> int` which returns the sum of the sizes of all the files present anywhere inside its argument. An empty directory has total size 0.
- Define `find : string -> fs -> bool` such that `find n f` returns **true** if a file named `s` exists in `f`, and **false** if no such file exists. (Only file names need to be considered, not directory names.)
- Define `names : fs -> string list` which returns the names of all files *and* directories inside its argument. The name of an entry must appear before the names of anything it contains.

6. Consider the following function:

```
let rec f l =
  match l with
  (* 1 *) | x :: _ :: t -> x :: f t
  (* 2 *) | x :: _ -> [x]
  (* 3 *) | [] -> []
```

- (a) What is the type of `f`?
- (b) What does `f [1;2;3;4;5]` evaluate to?
- (c) What does `f [1;2;3;4]` evaluate to?
- (d) Describe in at most one sentence what `f` computes.
- (e) Is `f` tail-recursive? Answer yes or no.
- (f) Will the OCaml compiler give an “exhaustivity” or an “unused case” warning for `f` as written above?

Each of the remaining parts suggests a modification to `f`. For each one, select exactly one of the following answers (just write the letter):

- A. The modified function compiles without any warning, and gives the same answer as the original function on all inputs.
- B. The modified function compiles without any warning, but gives a different answer from the original function on some input.
- C. The compiler gives an exhaustivity or an unused case warning.
- (g) We reorder the branches as: 2; then 1; then 3.
- (h) We reorder the branches as: 1; then 3; then 2.
- (i) We delete branch 2 altogether.
- (j) We replace branch 2 (in place) by `| [x] -> [x]`.
- (k) We replace branch 2 (in place) by `| - -> []`.
- (l) We replace branch 2 (in place) by `| - :: t -> t`.

7. **For this question you are not allowed to use recursion anywhere**, including in any helper function. You must use `List.fold_left` or `List.fold_right` (and you may use `List.map` and `@`).

Type information for your reference:

```
List.fold_left : ('a -> 'b -> 'a) -> 'a -> 'b list -> 'a
List.fold_right : ('a -> 'b -> 'b) -> 'a list -> 'b -> 'b
```

- (a) `forall : ('a -> bool) -> 'a list -> bool`, where `forall p l` is **true** iff all elements `e` of `l` satisfies `p e`.
 - (b) `last : 'a list -> 'a option`, which returns `Some e` where `e` is the last element of the list, and `None` for the empty list.
 - (c) `partition : ('a -> bool) -> 'a list -> 'a list * 'a list`, where `partition p l` returns a pair of lists (`yes`, `no`): `List yes` holds the elements of `l` satisfying `p` and `List no` holds the rest, both in their original relative order. For example,
`partition (fun x -> x mod 2 = 0) [1;2;3;4;5] = ([2;4], [1;3;5])`.
8. Write **tail-recursive** implementations of the following functions. You may use `List.rev`.

- (a) The function `zip` below is not tail-recursive. Rewrite it so that it is, without changing its behaviour on any input.

```
let rec zip l1 l2 =
  match (l1, l2) with
  | (x :: t1, y :: t2) -> (x, y) :: zip t1 t2
  | - -> []
```

- (b) Using the tree type from the lectures,

```
type 'a tree =
  | Leaf
  | Node of 'a tree * 'a * 'a tree
```

write a tail-recursive `count_greater : int -> int tree -> int`, where `count_greater n t` is the number of Nodes of `t` whose value is strictly greater than `n`. (*Hint: use an auxiliary function of type `int -> int tree list -> int` which keeps a list of the subtrees still to be visited.*)